

Struct

```
struct CuentaCD //El nombre es la ETIQUETA de la estructura
{
//miembros:
double saldo;
double tasa_interes;
int plazo; //meses hasta el vencimiento
};
```

Funciones básicas de una pila

```
#include <iostream>
using namespace std;

//Definición del struct
struct nodo_pila
{
    int dato;
    struct nodo_pila* link; //Al agregar un nuevo nodo se "anexa" a
nodo_pila
};
typedef nodo_pila Npila; //Para referir al tipo de dato nodo_pila* se
usará el sinónimo Npila

//Push -- agregar
void agregar(Npila* &pila, int n){ //Se pasa por referencia el puntero
al inicio de la pila porque será modificado
    Npila* aux = new(Npila); //Genero un nuevo nodo de forma dinámica
(new)
    aux -> dato = n; //Agrego el nuevo valor al campo "dato" de mi nuevo
nodo
    aux -> link = pila; //Hago que mi nuevo nodo apunte a donde está
apuntando el "índice" o "acceso" de la pila, es decir, al primer
elemento
    pila = aux; //Ahora mi primer elemento es el nuevo nodo ya que se
agrega "arriba" (LIFO), por lo que el "índice" o "acceso" de la pila
debe apuntar al nuevo nodo
}
```

```

//Get -- obtener
int obtener (Npila* &pila){ //Pasamos el indice como referencia ya que
al acceder a un dato este debe eliminarse
    int aux = pila -> dato; //Obtengo el último elemento almacenado en
la pila
    Npila* pila_aux = pila; //Necesito una variable auxiliar que
almacene temporalmente el valor de pila para no perder el nodo a
eliminar
    pila = pila -> link; //Apunto el "índice" al siguiente nodo (nodo
nro 2, al que apunta el nodo a eliminar)
    delete pila_aux; //Elimino el nodo que voy a mostrar ya que el
acceso es destructivo
    return aux;
}

//Preguntar si está vacía
bool isEmpty(Npila* pila){
    bool b = (pila == NULL) ? true : false;
    return b;
}

int main(void)
{
    Npila* pila = NULL;
    int n = 5;
    agregar(pila, n);
    cout<<obtener(pila)<<endl;
    cout<<isEmpty(pila);
}

```

Funciones básicas de una cola

```

#include <iostream>
using namespace std;

//Definir el struct
struct nodo_cola{
    int dato;
    struct nodo_cola* link;
};

typedef nodo_cola NCola;

```

```

//Preguntar si está vacía
bool isEmpty (NCola* frente){ //Al tener doble acceso con que cualquiera
de los dos (frente o fondo) esté vacío significa que la cola también lo
está
    bool b = (frente == NULL) ? true : false;
    return b;
}

//Push -- agregar
void agregar(NCola* &fondo, NCola* &frente, int n){ //Necesito el fondo
porque desde allí se agrega, pero también necesito el frente por si la
cola esta vacía
    NCola* aux = new (NCola); //Creo un nuevo nodo
    aux -> dato = n; //Agrego el valor al nuevo nodo
    aux -> link = NULL; //Al ser el último elemento apuntará a NULL
    if (fondo != NULL) fondo -> link = aux; // Si fondo != NULL
significa que apunta a algo (la cola no está vacía), por lo que el nodo
al que apunta fondo debe apuntar al nuevo nodo
    fondo = aux; //Luego, el fondo debe apuntar al nuevo nodo
    if (frente == NULL) frente = aux; //Si el frente es NULL la cola
está vacía, por lo que debo apuntar frente al nuevo nodo
}

//Get -- obtener
int obtener(NCola* &frente){ //Para sacar solo se necesita el frente
    bool b = isEmpty(frente); //Evalúo que la cola esté vacía
    if (b) return -1; //Si lo está retorno -1 (para manejo de errores)
    int aux = frente -> dato; //Obtengo el dato a mostrar
    NCola* cola_aux = frente; //Creo un puntero auxiliar que apunte al
mismo lugar que frente, es decir, al 1er nodo.
    frente = frente -> link; //Hago que frente apunte al siguiente nodo
    delete cola_aux; //Al ser de acceso destructivo debo eliminar el
nodo que obtengo
    return aux;
}

int main(void)
{
    NCola *cola_frente = NULL, *cola_fondo = NULL; }

```

Funciones básicas de una LSE

```
#include <iostream>
```

```

using namespace std;

//Defino el struct
struct nodo_lista_se{
    int dato;
    struct nodo_lista_se* link;
};

typedef nodo_lista_se NListaSE;

bool isEmpty(NListaSE* LSE){
    return (LSE == NULL) ? true : false;
}

//Imprimir todos los elementos de la lista
void imprimir(NListaSE* LSE){ //Paso por parámetro el puntero al primer
    elemento de la lista
    NListaSE* LSE_aux = LSE; //Creo un puntero auxiliar para poder
    avanzar entre los nodos
    while (LSE_aux != NULL){ //Mientras que mi puntero sea != NULL estoy
    recorriendo los nodos, cuando sea NULL es porque ya pasé el último
        cout << (LSE_aux -> dato) << endl; //Imprimo el dato del nodo
        LSE_aux = LSE_aux -> link; //Me muevo al siguiente nodo
    }
}

//Agregar elemento al principio
void agregar_inicio(NListaSE* &LSE, int n){
    NListaSE* aux = new (NListaSE); //Creo el nuevo nodo
    aux -> dato = n; //Almaceno el dato
    aux -> link = LSE; //Hago que mi nuevo nodo apunte al primer nodo,
    es decir, a donde apunta LSE (puntero inicial)
    LSE = aux; //Hago que mi puntero inicial, LSE, apunte al nuevo nodo
}

//Agregar elemento al final
void agregar_final(NListaSE* &LSE, int n){
    NListaSE* aux = new (NListaSE); //Creo el nuevo nodo
    aux -> dato = n; //Almaceno el nuevo valor
    aux -> link = NULL; //Como lo agrego al final el nuevo nodo debe
    apuntar a NULL
    NListaSE* LSE_aux = LSE; //Puntero auxiliar para poder recorrer la
    lista
    //Evaluar si esta vacía
    isEmpty(LSE_aux);
}

```

```

//Recorrer la lista hasta el último elemento
while((LSE_aux -> link) != NULL){
    LSE_aux = LSE_aux -> link; //Avanzo hasta el último elemento
}
LSE_aux -> link = aux; //Hago que el último nodo de la lista apunte
al nuevo nodo
}

```

//Agregar elemento al final forma 2: Utilizar dos punteros (actual y anterior)

```

void agregar_final_2 (NListaSE *&LSE, int n){
    NListaSE* aux = new(NListaSE); //Creo el nuevo nodo
    aux -> dato = n; //Almaceno el nuevo valor
    aux -> link = NULL; //Como lo agrego al final el nuevo nodo debe
apuntar a NULL
    NListaSE* actual = LSE;
    NListaSE* anterior = NULL;
    //Evaluar si está vacía
    if (!isEmpty(LSE)){ //Si no está vacía
        //Recorrer la lista hasta el último elemento
        while ((actual != NULL)){
            anterior = actual; //Muevo anterior en una posición
            actual = actual -> link; //Muevo actual a la siguiente
posición
        }
        anterior -> link = aux; //Hago que el último nodo de la lista
apunte al nuevo nodo
    }
    else LSE = aux; //Si la lista está vacía LSE debe apuntar al nuevo
nodo
}

```

//Eliminar primer coincidencia de un elemento de la lista (Rehacer haciendo el cambio del while)

```

void borrar(NListaSE* &LSE, int n){
    NListaSE* actual = LSE;
    NListaSE* anterior = NULL; //Puntero anterior atrasado en una
posición con actual

    //Recorro la lista buscando el elemento
    while((actual != NULL) && ((actual -> dato) != n)){ //Mientras que
actual sea != NULL aún recorro elementos de la lista, mientras que
actual != n aún no encuentro el elemento
        anterior = actual; //Muevo anterior en una posición
        actual = actual -> link; //Muevo actual a la siguiente posición
    }
}

```

```

    }
    //Al salir del while hay dos opciones:

    //1. El elemento buscado es el primer elemento
    if ((anterior == NULL) && (actual != NULL)){ //En este caso anterior
no avanzó porque el elemento buscado es el primero y si actual != NULL
es que seguimos dentro de la lista
        LSE = actual -> link; //Hago que mi puntero índice apunte al
segundo nodo
        delete (actual); //Elimino el primer nodo, donde se halla el
elemento buscado
    }

    //2. El elemento buscado no es el primer elemento
    else if((anterior != NULL) && (actual != NULL)){ //En este caso
anterior deja de ser NULL, por lo que el elemento no está en la primera
posición y si actual != NULL es que seguimos dentro de la lista
        anterior -> link = actual -> link; //Necesito que el nodo
anterior al que voy a eliminar se conecte al nodo siguiente al que voy a
eliminar
        delete (actual); //Elimino el nodo de interés
    }
}

int main() {
    NListaSE* lista = NULL;
    agregar_final_2(lista, 10); // Lista: [10]
    agregar_final_2(lista, 20); // Lista: [10] -> [20]
    imprimir(lista);
    return 0;
}

```